# Documento de diseño — Club Deportivo Séneca

Proyecto 2  ISIS-1226 Diseño y Programación Orientada a Objetos

Samuel Beltran (202522601)  Rafael Esteban Mora Wilches (202511391)  Daniela Sanchez (202420100)

---

Este documento describe cómo está construida la aplicación del Proyecto 2. Qué debe hacer está en el documento de historias de usuario del grupo, aquí se explica qué clases y métodos resuelven cada historia.

Los diagramas que aparecen en el documento están también en la carpeta Diagramas/ del repositorio, cada uno con su archivo fuente (.dot o .mmd) y sus imágenes (.png y .svg).

### 1. Alcance

El Proyecto 2 parte de la entrega del Proyecto 1 y le agrega lo que pide el enunciado:

- **Torneos internos** recreativos y competitivos, de deportes de conjunto (se inscriben equipos completos) o de disciplinas individuales (se inscriben jugadores, o parejas si el torneo es de dobles).
- **Inscripción y retiro** antes de que el torneo empiece. Al retirarse se libera el cupo.
- **Cupos preferenciales:** el 20 % de los cupos, redondeado hacia arriba, queda para los mejores del ranking. Si no se llenan antes de la fecha límite, pasan a ser regulares.
- **Participación de empleados** en torneos individuales, siempre que no estén de turno ese día. No pagan la inscripción del competitivo, pero tampoco reciben el premio en efectivo.
- **Participantes externos y estudiantes** de la universidad.
- **Consulta del estado y del historial** de cada torneo: encuentros, participantes y posiciones.
- **Premios:** un bono de descuento para el ganador de un torneo recreativo, que se puede guardar y usar en una compra futura y no se acumula con otros descuentos, y un premio en efectivo para el ganador de un competitivo.
- **Tres aplicaciones de consola**, una por tipo de usuario, con inicio de sesión obligatorio y carga y guardado automáticos de los datos.

### 2. Historias de usuario y dónde se resuelven

Las historias vienen del banco oficial del Proyecto 1 y de las historias nuevas de torneos que redactó el grupo.

| Historia | Clases y métodos principales | Consola | Pruebas |
|---|---|---|---|
| HU-ADM-02 Revisar cambios de turno | ClubDeportivo.aprobarCambioTurno, rechazarCambioTurno, verificarCobertura | Administrador › Turnos | HU_ADM_02_CambioTurnoTest |
| HU-ENT-02 Mínimo de jugadores habilitados | Equipo.getJugadoresNoDisponibles, ClubDeportivo.verificarMinimoHabilitados | Entrenador › Revisar el mínimo | HU_ENT_02_MinimoHabilitadosTest |

| | | | |
|---|---|---|---|
| HU-SOC-01 Reservar instalación | ClubDeportivo.reservar, Instalacion.motivoRechazo, CanchaPadel | Participante (socio) › Reservar | HU_SOC_01_ReservaTest, ConsultasConsolaTest |
| HU-SOC-02 Inscribirse en un equipo | ClubDeportivo.inscribirJugadorEnEquipo, Equipo.inscribirJugador | Participante (socio) › Inscribirme en un equipo | HU_SOC_02_InscripcionEquipoTest |
| HU-TOR-ADM-01 Crear torneo | ClubDeportivo.crearTorneoIndividual, crearTorneoConjunto, registrarTorneo, Torneo.validarDatos | Administrador › Torneos › Crear | HU_TOR_ADM_01_CrearTorneoTest, TorneoValidacionTest |
| HU-TOR-ADM-02 Finalizar y entregar premios | Torneo.iniciar, registrarEncuentro, finalizar, calcularBolsa | Administrador › Torneos › Iniciar / Encuentro / Finalizar | HU_TOR_ADM_02_FinalizarTest, TorneoCicloTest |
| HU-TOR-ADM-03 y HU-TOR-SOC-05 Consultar torneos | ClubDeportivo.consultarTorneos, resumenTorneos, consultarHistorialTorneo, Torneo.generarHistorial | Las tres consolas › Torneos › Consultar / Historial | HU_TOR_ADM_03_ConsultaTest, HU_TOR_SOC_05_ConsultaTest |
| HU-TOR-SOC-01 Inscribirse en un torneo | ClubDeportivo.inscribirEnTorneoIndividual, inscribirEquipoEnTorneo, puedeGestionarEquipo | Participante › Torneos › Inscribirme / Inscribir a mi equipo | HU_TOR_SOC_01_InscripcionTest, TorneoCuposTest |
| HU-TOR-SOC-02 Salirse del torneo | ClubDeportivo.salirDeTorneo, retirarEquipoDeTorneo, Torneo.retirar | Participante › Torneos › Salirme / Retirar a mi equipo | HU_TOR_SOC_02_DesinscribirseTest |
| HU-TOR-SOC-03 Cupos preferenciales | Torneo.getCuposPreferenciales, getCuposDisponibles, preferencialesLiberados; ClubDeportivo.calcularRankingIndividual, calcularPuntosInternos, rankingsDeCategoria, | Automático al inscribirse | HU_TOR_SOC_03_CuposPreferencialesTest, TorneoCuposTest |

| | | | |
|---|---|---|---|
| | esEleglePreferencial | | |
| HU-TOR-SOC-04 Usar el bono | BonoDescuento, Venta.aplicarBono, ClubDeportivo.aplicarBono | Participante › Tienda, cafetería y bonos | HU_TOR_SOC_04_UsarBonoTest, BonoDescuentoTest, VentaBonoTest |
| HU-TOR-EMP-01 Empleados en torneos | Empleado.tieneTurnoEl, declararNivel; InscripcionTorneo.esDeEmpleado | Entrenador › Torneos › Inscribirme | HU_TOR_EMP_01_EmpleadosTest |
| Registro de externos y estudiantes | ClubDeportivo.registrarExterno, ParticipanteExterno | Participante › crear cuenta | RegistroExternoTest |

### 3. Arquitectura general

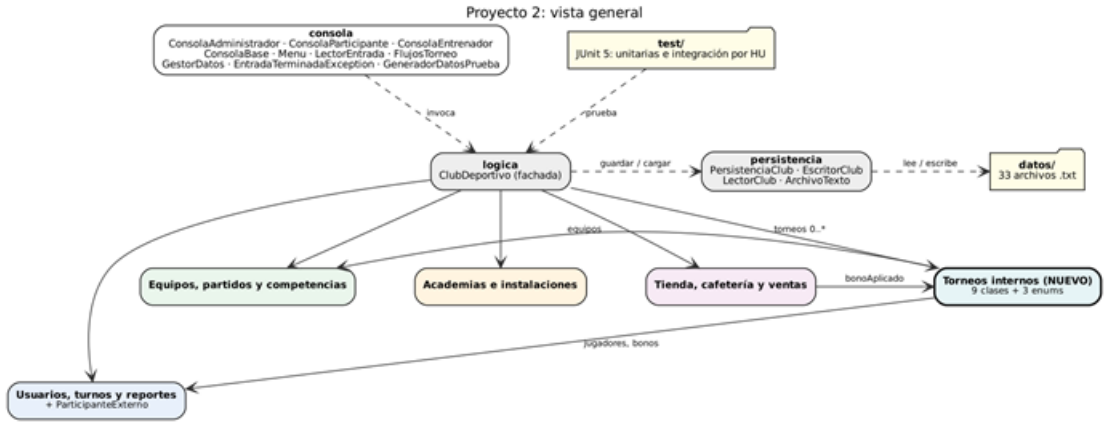



Figura 1. Vista general (Diagramas/clases/vista_general_p2)

La aplicación está organizada en capas. Cada capa solo usa a la que está debajo:

| Capa (paquete) | Responsabilidad | Clases |
|---|---|---|
| consola | Interfaz con el usuario: menús, lectura y validación de datos, mensajes. | ConsolaBase, ConsolaAdministrador, ConsolaEntrenador, ConsolaParticipante, FlujosTorneo, Menu, LectorEntrada, EntradaTerminadaException, GestorDatos, GeneradorDatosPrueba |
| logica | Fachada del sistema: único punto de entrada a las reglas del negocio. | ClubDeportivo |
| modelo.* | Las clases del dominio y sus reglas. | usuarios, torneos (nuevo), conjunto, competencias, academias, entrenamiento, ventas, reportes, enums |
| persistencia | Guardar y cargar el club en archivos de texto. | PersistenciaClub, EscritorClub, LectorClub, ArchivoTexto |

| | | |
|---|---|---|
| excepciones | Error de regla del negocio. | ReglaNegocioException |

**Principios que se siguieron:**

- **Fachada.** La consola nunca crea objetos del modelo ni cambia sus listas: todo pasa por ClubDeportivo.
- **Reglas en el modelo.** Cada regla vive en la clase que tiene la información. Por ejemplo, los cupos los decide Torneo, si un empleado está de turno lo decide Empleado y si un bono se puede usar lo decide Venta. Cuando una regla no se cumple, el modelo lanza ReglaNegocioException con un mensaje claro.
- **Herencia donde hay un "es un" real.** TorneoIndividual y TorneoConjunto son un Torneo. InscripcionTorneoIndividual e InscripcionTorneoEquipo son una InscripcionTorneo. ParticipanteExterno es un Usuario.
- **Capas sin dependencias hacia arriba.** El modelo no conoce la consola ni la persistencia.

**4. Cambios respecto al Proyecto 1**

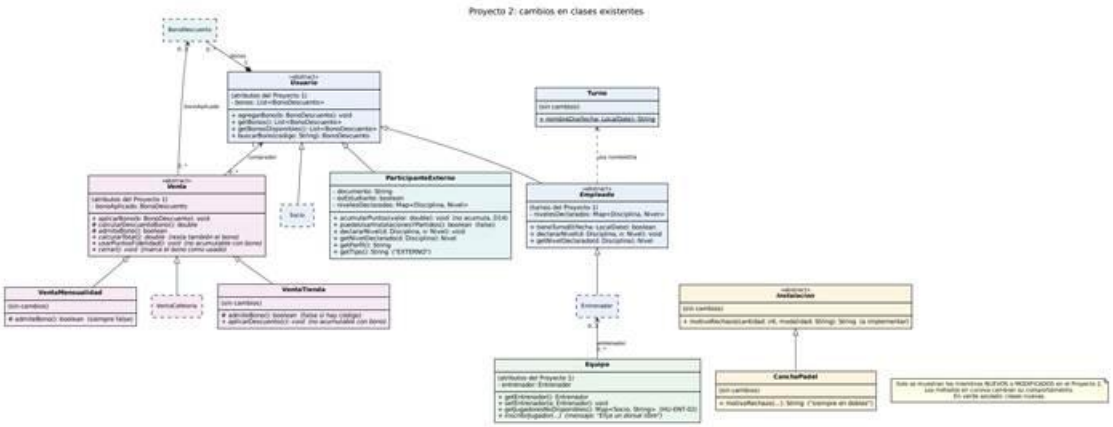



Figura 2. Cambios en clases existentes (Diagramas/clases/detallado_cambios_p2)

El diagrama solo muestra los atributos y métodos nuevos o modificados.

| Clase | Cambio | Por qué |
|---|---|---|
| Usuario | Nueva lista bonos con agregarBono, getBonosDisponibles y buscarBono | Cualquier usuario puede ganar y guardar bonos (HU-TOR-SOC-04). |
| ParticipanteExterno (**nueva**) | documento, esEstudiante, nivelesDeclarados. No acumula puntos (decisión D14) y no reserva ni juega partidos | El enunciado permite que participen estudiantes y personas externas. |
| Empleado | nivelesDeclarados, declararNivel, getNivelDeclarado y tieneTurnoEl(fecha) | Un empleado solo se inscribe si no está de turno el día del torneo (HU-TOR-EMP-01) y necesita un nivel para |

| | | |
|---|---|---|
| | | entrar a un torneo individual. |
| Equipo | Atributo entrenador y método getJugadoresNoDisponibles() | El entrenador puede inscribir a su equipo, y HU-ENT-02 necesita saber quién no está habilitado y por qué. |
| Venta | Atributo bonoAplicado y métodos aplicarBono, calcularDescuentoBono y admiteBono. calcularTotal resta el bono, cerrar lo marca como usado y usarPuntosFidelidad lo rechaza si ya hay bono | Usar el bono en una compra sin acumularlo con otros descuentos. |
| VentaTienda / VentaMensualidad | admiteBono(): la tienda no lo admite si ya hay un código de empleado y la mensualidad nunca lo admite | Regla de “no acumulable”. |
| Instalacion / CanchaPadel | motivoRechazo(cantidad, modalidad). El pádel solo se juega en dobles | Mensajes claros en HU-SOC-01. |
| ClubDeportivo | Torneos, rankings, externos, bonos, % del premio, HU-ENT-02 y consultas para las consolas (sección 6) | Punto de entrada de toda la funcionalidad nueva. |
| persistencia | Seis archivos nuevos y campos nuevos en tres archivos existentes. La clase se dividió en cuatro (sección 9) | Guardar todo lo nuevo. |
| consola | Reescrita: tres aplicaciones con una clase padre común (sección 8) | Actividad 3 del enunciado. |

**5. Diseño de los torneos internos**

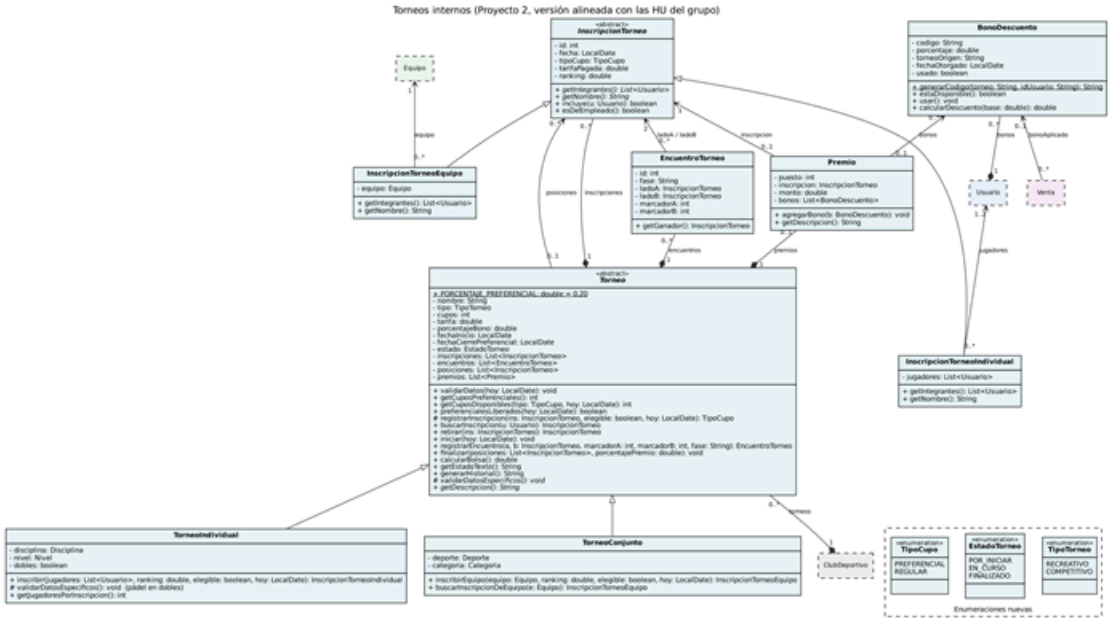


Figura 3. Torneos internos (Diagramas/clases/detallado_torneos)

### 5.1 Clases

| Clase | Responsabilidad |
|---|---|
| Torneo (abstracta) | Datos comunes (nombre, tipo, cupos, tarifa, % del bono, fechas y estado), las inscripciones, los encuentros, las posiciones y los premios. Decide los cupos, el ciclo de vida y los premios. |
| TorneoIndividual | Disciplina, nivel y si es de dobles. Inscribe a uno o dos jugadores. Valida que el pádel sea en dobles. |
| TorneoConjunto | Deporte y categoría. Inscribe un equipo completo y lo busca por equipo. |
| InscripcionTorneo (abstracta) | Id, fecha, tipo de cupo, tarifa pagada y ranking. Sabe quiénes la integran y si es “de empleado”. |
| InscripcionTorneoIndividual | Uno o dos jugadores (Usuario). |
| InscripcionTorneoEquipo | Un Equipo. Sus integrantes son los jugadores activos del equipo. |
| EncuentroTorneo | Un partido del torneo, con su fase y marcador. Sirve para el historial. |
| Premio | Puesto, inscripción ganadora, dinero entregado y bonos entregados. |
| BonoDescuento | Código único, % de descuento, torneo de origen, fecha y si ya se usó. |
| Enumeraciones nuevas | TipoTorneo (RECREATIVO, COMPETITIVO), EstadoTorneo (POR_INICIAR, EN_CURSO, FINALIZADO) y TipoCupo (PREFERENCIAL, REGULAR). |

### 5.2 Ciclo de vida de un torneo

POR_INICIAR ──iniciar()──► EN_CURSO ──finalizar(posiciones)──► FINALIZADO
(inscripciones abiertas) (encuentros) (premios entregados)

| Estado | Qué se permite | Quién lo controla |
|---|---|---|
| POR_INICIAR | Inscribirse y retirarse. | Torneo.registrarInscripcion y Torneo.retirar rechazan si el torneo ya no está abierto. |
| EN_CURSO | Registrar encuentros. Para iniciar se necesitan al menos 2 inscritos. | Torneo.iniciar y Torneo.registrarEncuentro. |
| FINALIZADO | Solo consultar. No se puede finalizar dos veces. | Torneo.finalizar. |

Se puede consultar el estado y el historial de un torneo en cualquier momento (generarHistorial).

### 5.3 Validación al crear un torneo (Torneo.validarDatos)

- El nombre es obligatorio y **único** en el club. Esto lo revisa ClubDeportivo.registrarTorneo.

- Los cupos deben ser mayores que 0.
- **Competitivo:** la tarifa es obligatoria (mayor que 0) y no hay bono.
- **Recreativo:** no se cobra tarifa y el bono debe estar entre 1 % y 100 %.
- La fecha de inicio no puede haber pasado. La fecha límite de los preferenciales debe ser anterior a la de inicio.
- TorneoIndividual: el pádel siempre es en dobles.

**5.4 Cupos preferenciales (HU-TOR-SOC-03)**

- getCuposPreferenciales() devuelve el **20 % de los cupos redondeado hacia arriba**. Por ejemplo, de 8 cupos, 2 son preferenciales y 6 regulares.
- Mientras no llegue la fecha límite, se cuentan aparte los preferenciales y los regulares. Desde la fecha límite (preferencialesLiberados(hoy)), todos los cupos libres son regulares.
- Al inscribirse, la fachada calcula el **ranking** del participante y si es **elegible** (esElegiblePreferencial). Es elegible si su ranking está entre los N mejores de la categoría, donde N es el número de cupos preferenciales.
  - Ranking individual: el ELO de la academia en esa disciplina, o el promedio de la pareja en dobles. Los externos, los estudiantes y los empleados sin perfil de jugador no tienen ranking (-1) y nunca toman un preferencial.
  - Ranking de un equipo: puntos en partidos internos (victoria 3, empate 1, derrota 0, según el deporte).
- Torneo.registrarInscripcion asigna un **PREFERENCIAL** si la inscripción es elegible y queda uno libre. Si no, asigna un **REGULAR**. Si solo quedan preferenciales y el participante no es elegible, se rechaza con el mensaje “reservados para los mejores del ranking”.

**5.5 Tarifas y premios (HU-TOR-ADM-02)**

- **Tarifa pagada:** en un competitivo se paga la tarifa, salvo que **todos** los integrantes sean empleados (esDeEmpleado()). En un recreativo siempre es 0.
- **Recreativo:** cada integrante del primer puesto recibe un BonoDescuento, empleados incluidos. El código es BONO-<TORNEO>-<idUsuario>.
- **Competitivo:** la bolsa es la suma de las tarifas pagadas (calcularBolsa). El premio es la bolsa por el % configurable del club (PREMIO en config.txt, 70 % por defecto). Lo recibe el **primer puesto que no sea de empleado**.
- El administrador registra las posiciones en orden. Las posiciones deben ser inscripciones distintas del mismo torneo.

**5.6 Uso del bono (HU-TOR-SOC-04)**

- BonoDescuento.estaDisponible() es verdadero mientras no se haya usado. calcularDescuento(base) aplica el %.
- Venta.aplicarBono rechaza la operación si:
  - la venta está cerrada;
  - el bono no existe o ya se usó;
  - ya hay otro bono aplicado;
  - ya se usaron puntos o un código de empleado (admiteBono).
- La mensualidad no admite bonos.
- Al confirmar la venta, cerrar() marca el bono como usado.

## 6. La fachada ClubDeportivo




Figura 4. Métodos nuevos de la fachada (Diagramas/clases/detallado_fachada_p2)

ClubDeportivo guarda las colecciones del club (usuarios, equipos, torneos, ventas…) y ofrece un método por cada acción que hacen las consolas. Busca los objetos por nombre o id, revisa permisos y delega las reglas en el modelo.

**Métodos nuevos, por grupo:**

| Grupo | Métodos |
|---|---|
| Crear y consultar torneos | crearTorneoIndividual, crearTorneoConjunto, registrarTorneo, buscarTorneo, consultarTorneos(estado), resumenTorneos, consultarHistorialTorneo, consultarTorneosDe(usuario) |
| Rankings y cupos preferenciales | obtenerNivel, calcularRankingIndividual, calcularPuntosInternos, rankingsDeCategoria (individual y conjunto), esElegiblePreferencial |
| Inscripción y retiro | inscribirEnTorneoIndividual, puedeGestionarEquipo, inscribirEquipoEnTorneo, salirDeTorneo, retirarEquipoDeTorneo |
| Ciclo del torneo | iniciarTorneo, registrarEncuentroTorneo, finalizarTorneo, get/setPorcentajePremioCompetitivo |
| Externos y bonos | registrarExterno, buscarUsuarioPorLogin, aplicarBono |
| Equipos y turnos | verificarMinimoHabilitados, getEquiposDe(entrenador), asignarEntrenador, getSolicitudesPendientes, rechazarCambioTurno |
| Consultas y acciones para las consolas | getEquiposDeSocio, getEntrenadores, getReservasDe, getSolicitudesDe, declararNivel, reservar(..., hoy) |

**Ejemplo: inscripción en un torneo individual** (inscribirEnTorneoIndividual), resumido:

1. Busca el torneo. Rechaza si el usuario es administrador o si el torneo es de conjunto.

2. Arma la lista de jugadores. En dobles agrega a la pareja, que debe estar registrada.
3. Por cada jugador revisa que tenga el **nivel** del torneo y que, si es empleado, **no esté de turno** ese día.
4. Calcula el ranking de la inscripción y si es elegible para un cupo preferencial.
5. Llama a TorneoIndividual.inscribir, que asigna el cupo y la tarifa.

## 7. Diagramas de secuencia

### 7.1 Inscripción en un torneo individual (HU-TOR-SOC-01, SOC-03 y EMP-01)

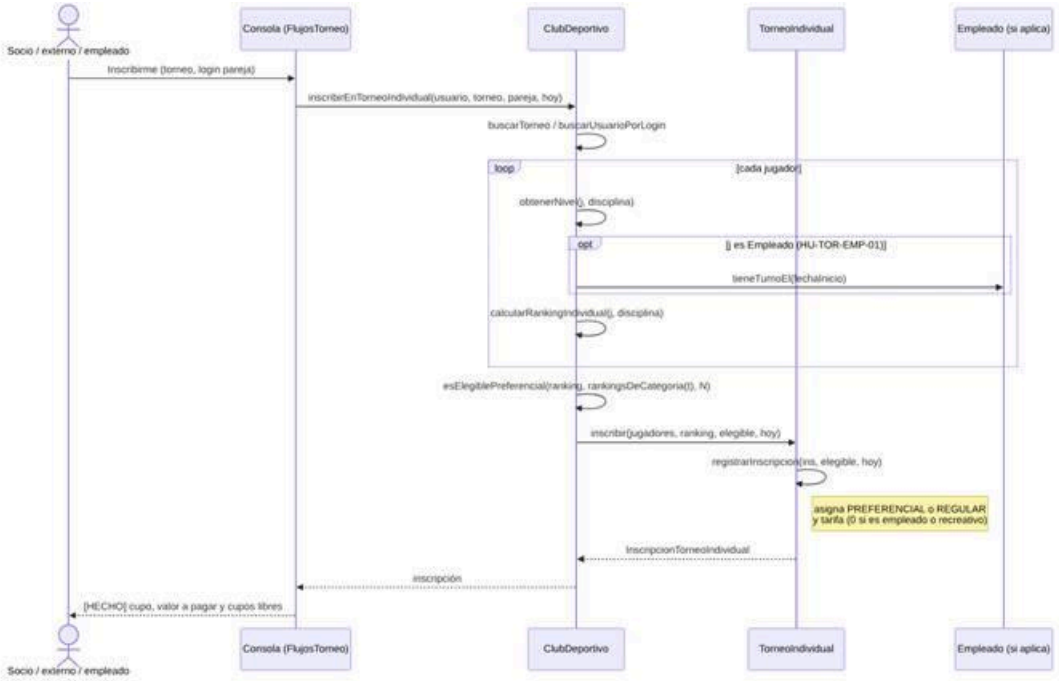



Figura 5. Inscripción en torneo (Diagramas/secuencias/sec5_inscripcion_torneo)

### 7.2 Finalizar un torneo y entregar premios (HU-TOR-ADM-02)

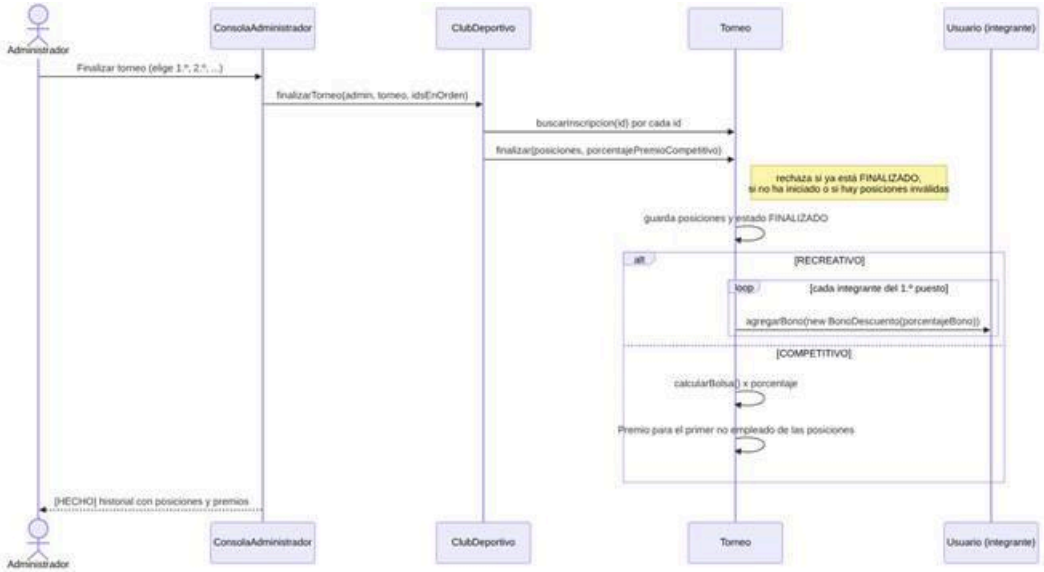


Figura 6. Finalizar torneo (Diagramas/secuencias/sec6_finalizar_torneo)

### 7.3 Ciclo de vida de una aplicación de consola

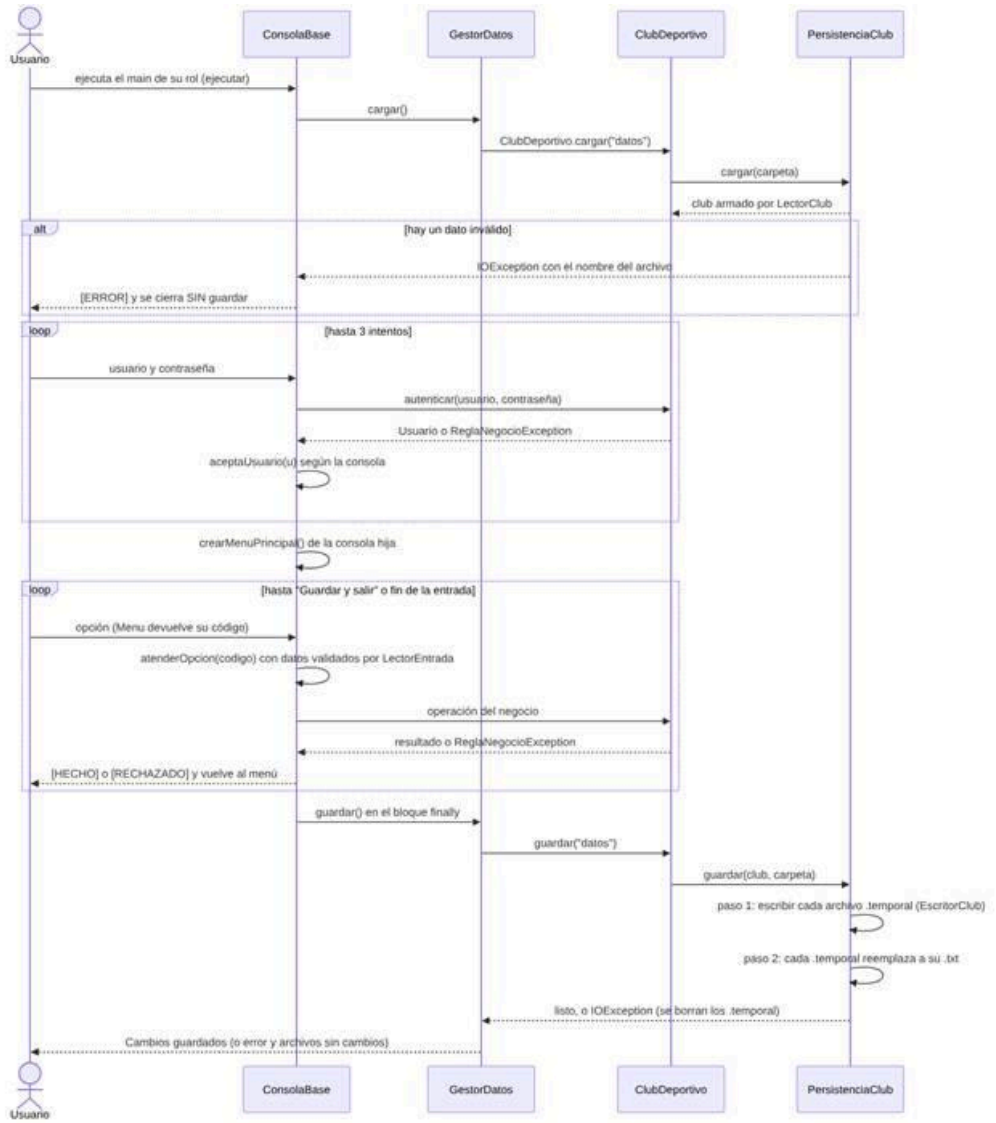



Figura 7. Ciclo de la consola (Diagramas/secuencias/sec7_ciclo_consola)

## 8. Interfaz de consola

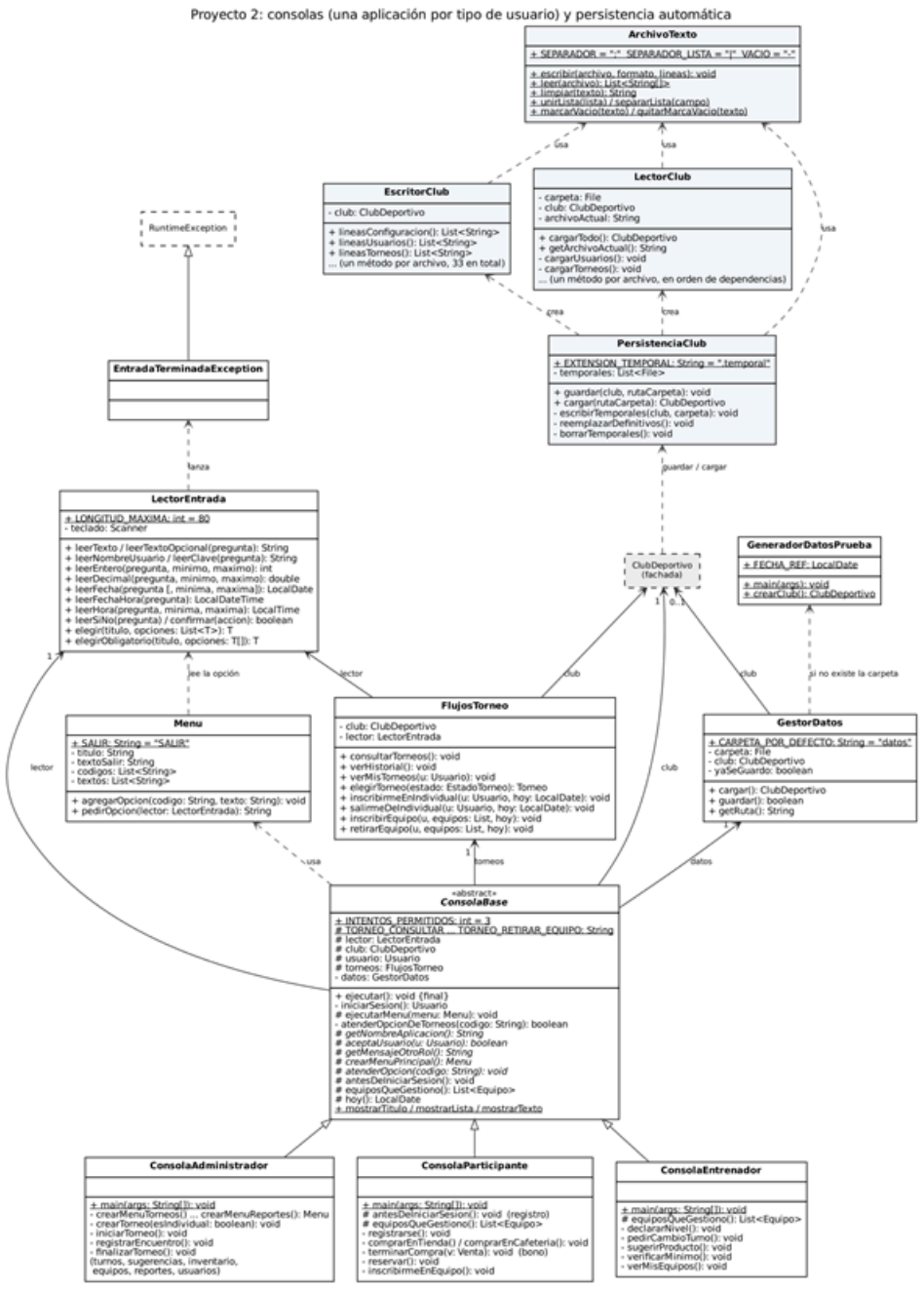



Figura 8. Consolas y persistencia (Diagramas/clases/detallado_consolas_p2)

### 8.1 Tres aplicaciones, una clase padre

| Aplicación (main) | Usuarios aceptados | Usuarios de prueba |
|---|---|---|
| ConsolaAdministrador | Administrador | admin / admin123 |
| ConsolaEntrenador | Entrenadores y fisioterapeutas | cmendez / ent123, prios / fis123 |
| ConsolaParticipante | Socios, estudiantes y externos (puede crear una cuenta antes de entrar) | s001 / 1234, jospina / 1234, rvelez / 1234 |

ConsolaBase.ejecutar() es final y siempre sigue los mismos pasos:

1. Cargar los datos.
2. Iniciar sesión, con máximo 3 intentos, revisando las credenciales contra los usuarios del modelo y que el tipo de usuario pueda usar esa aplicación.
3. Mostrar el menú del rol.
4. Guardar en un bloque finally.

Así ninguna aplicación se puede saltar el inicio de sesión ni el guardado. Las consolas hijas solo definen aceptaUsuario, crearMenuPrincipal y atenderOpcion.

### 8.2 Clases que evitan las clases gigantes

| Clase | Qué aporta |
|---|---|
| Menu | Cada opción tiene un **código** y un texto. pedirOpcion devuelve el código y la consola lo atiende con un switch. Así se agregan opciones solo para algunos usuarios sin desordenar los números. |
| LectorEntrada | **Toda** la validación de entradas en un solo lugar, que vuelve a preguntar hasta recibir un valor válido. Valida enteros y decimales con rango, fechas reales con límites, horas, textos sin ; ni \| y con máximo 80 caracteres, nombres de usuario sin espacios, sí/no y opciones de una lista. |
| FlujosTorneo | Los pasos de torneos que comparten varias consolas: consultar, ver historial, inscribirse, salirse, inscribir o retirar un equipo. |
| GestorDatos | Carga y guarda automáticamente, sin pedir nombres de archivo. |
| ConsolaBase.ejecutarMenu | Maneja los errores de **todas** las opciones en un solo lugar: muestra [RECHAZADO] motivo y vuelve al menú. |

### 8.3 Separación entre interfaz y lógica

La consola solo filtra lo que tiene sentido mostrar: torneos en el estado correcto, el participante B distinto del A, la pareja distinta de uno mismo, fechas y horas dentro de rangos posibles. **Las reglas del negocio están solo en**

**el modelo**, entre ellas el nombre de torneo único y la reserva en una fecha que ya pasó. Las acciones que no se pueden deshacer, como iniciar o finalizar un torneo, salirse o retirar un equipo, piden confirmación.

### 8.4 Validación de entradas

Todos los métodos de LectorEntrada **vuelven a preguntar** hasta que el valor sea válido, así que las consolas nunca reciben datos fuera de rango ni con un formato equivocado.

| Método | Qué revisa |
|---|---|
| leerTexto, leerTextoOpcional | Quita los espacios de los lados. El texto es obligatorio (en el opcional, Enter lo deja vacío). No acepta ; ni \| porque son los separadores de los archivos. Máximo 80 caracteres. |
| leerNombreUsuario, leerClave | Usuario sin espacios. Contraseña obligatoria. |
| leerEntero(min, max) | Solo números enteros, dentro del rango. Rechaza números demasiado grandes. |
| leerDecimal(min, max) | Acepta coma o punto. Solo acepta dígitos, así que rechaza NaN, Infinity y 5d. Revisa el rango. |
| leerFecha([min, max]) | Formato aaaa-mm-dd. Rechaza fechas que no existen, como 2026-02-30, y puede tener límites (por ejemplo, “a partir de mañana”). |
| leerFechaHora, leerHora(min, max) | Formato aaaa-mm-dd hh:mm. Las horas en punto quedan dentro del horario del club. |
| leerSiNo, confirmar | Solo acepta s, si, sí, n o no. confirmar se usa antes de acciones que no se pueden deshacer. |
| elegir, elegirObligatorio | Opciones numeradas con rango validado. elegir permite cancelar con 0 y avisa si la lista está vacía. elegirObligatorio se usa con los valores de un enum. |

### 8.5 Cómo extender la interfaz

- **Nueva opción en un rol:** agregar menu.agregarOpcion("CODIGO", "Texto"), un case "CODIGO": en atenderOpcion y un método privado que haga el trabajo.
- **Nueva opción que comparten varios roles:** agregar el método en FlujosTorneo (o en otra clase de pasos), una constante de código en ConsolaBase y su case en atenderOpcionDeTorneos.
- **Nueva aplicación o nuevo tipo de usuario:** crear class ConsolaX extends ConsolaBase con su main y los métodos abstractos. La carga, el inicio de sesión, el manejo de errores y el guardado se heredan.

## 9. Persistencia

### 9.1 Funcionamiento

- **Al iniciar** cualquier aplicación se carga la carpeta datos/. Si no existe, se crea con los datos de prueba.
- **Al terminar** (opción "Guardar y salir", fin de la entrada o un error inesperado) se guarda en **los mismos archivos**.
- Si la carga falla, **no se guarda**, para no borrar los datos reales.
- El enunciado no exige guardar después de cada acción, así que se guarda al terminar el programa.

**9.2 Clases**

| Clase | Responsabilidad |
|---|---|
| PersistenciaClub | Organiza el guardado y la carga. La fachada la usa con guardar(club, carpeta) y cargar(carpeta). |
| EscritorClub | Un método por archivo (33 en total) que convierte el club en líneas de texto. |
| LectorClub | Lee los archivos en **orden de dependencias** y arma el club de nuevo. Si un dato está mal escrito, el error dice en qué archivo está. |
| ArchivoTexto | Leer, escribir, limpiar textos, unir y separar listas, y la marca de vacío. |

**Guardado "todo o nada":** primero se escribe cada archivo como .temporal. Solo si todos quedaron bien, cada temporal reemplaza a su .txt. Si algo falla, se borran los temporales y los archivos anteriores no cambian.

**9.3 Formato y archivos nuevos**

Los archivos son texto UTF-8, con ; entre campos, | entre los elementos de una lista y - para un campo vacío. La primera línea de cada archivo es un comentario # que explica los campos. Las referencias entre objetos se guardan por id o por nombre.

| Archivo | Contenido nuevo del Proyecto 2 |
|---|---|
| torneos.txt | Clase, tipo, cupos, tarifa, % del bono, fechas, estado, posiciones y disciplina o deporte. |
| inscripciones_torneo.txt | Tipo de cupo, tarifa pagada, ranking y equipo o jugadores. |
| encuentros_torneo.txt | Fase y marcador de cada encuentro. |
| premios_torneo.txt | Puesto, inscripción ganadora, dinero y códigos de los bonos. |
| bonos.txt | Bonos de cada usuario y si ya se usaron. |
| niveles_declarados.txt | Niveles declarados por externos y empleados. |
| usuarios.txt | Nuevo tipo EXTERNO (documento y si es estudiante). |
| config.txt | Nueva clave PREMIO (% para el premio en efectivo). |
| equipos.txt, ventas.txt | Nuevos campos: entrenador del equipo y bono aplicado. |

**Orden de carga:** configuración → instalaciones → usuarios → niveles y bonos → turnos y códigos → equipos → competencias → partidos → academias → sesiones → solicitudes → tienda y ventas → sugerencias → torneos, inscripciones, posiciones, encuentros y premios. Primero se carga lo que otros archivos necesitan.

**10. Manejo de errores**

| Situación | Quién la detecta | Qué pasa |
|---|---|---|
| Se viola una regla del negocio | El modelo lanza ReglaNegocioException | La consola muestra [RECHAZADO] motivo y vuelve al menú. |
| Dato con formato equivocado | LectorEntrada | Explica el formato esperado y vuelve a preguntar. |
| Error inesperado dentro de una opción | ConsolaBase.ejecutarMenu | Muestra [ERROR] ... y vuelve al menú. |
| Se acaba la entrada (Ctrl+D / Ctrl+Z) | LectorEntrada lanza EntradaTerminadaException | La aplicación guarda y se cierra. |
| Archivo de datos dañado | PersistenciaClub.cargar | Indica el archivo con el problema y se cierra sin sobrescribir nada. |
| No se puede guardar | PersistenciaClub.guardar | Muestra el error y deja los archivos anteriores sin cambios. |

**11. Pruebas**

Las pruebas usan **JUnit 5** y un club pequeño en memoria con fechas fijas (test/pruebas/ClubDePrueba). Así cada prueba empieza desde el mismo estado y no depende de la carpeta datos/. **Pasan las 125 pruebas.**

| Carpeta | Tipo | Clases (pruebas) |
|---|---|---|
| test/modelo/torneos | Unitarias | TorneoCuposTest (16), TorneoCicloTest (10), TorneoValidacionTest (9), BonoDescuentoTest (3) |
| test/modelo/ventas | Unitarias | VentaBonoTest (8) |
| test/integracion | Integración, al menos una clase por historia, con un método por criterio de aceptación (CA1_...) | 14 clases, 68 pruebas |
| test/persistencia | Integración | PersistenciaProyecto2Test (2): guardar y cargar torneos, bonos y externos |
| test/logica | Unitarias de la fachada | ConsultasConsolaTest (9): consultas y acciones que usan las consolas |

Las pruebas de integración se escribieron a partir de los criterios de las historias, antes que la implementación (TDD).

**Pruebas manuales de la consola.** Como dice la actividad 2d, la interfaz no tiene pruebas automáticas. Las tres aplicaciones se probaron a mano con entradas escritas de antemano, sobre una copia de datos/. Se revisó lo siguiente:

- **Inicio de sesión:** se rechazan una contraseña incorrecta y un usuario de otro rol, y la aplicación se cierra después de 3 intentos.
- **Validación de entradas:** se rechazan textos en campos numéricos, números fuera de rango, NaN, 5d, fechas que no existen o fuera de rango y nombres de usuario con espacios.
- **Creación de cuenta:** se crea la cuenta de un externo, volviendo a pedir la contraseña cuando no coincide.
- **Torneos:**
  - inscripción y salida de un torneo, con el cupo liberado;
  - inscripción de un equipo en un cupo preferencial;
  - nombre de torneo repetido, rechazado por el modelo;
  - empleado de turno el día del torneo, rechazado por el modelo;
  - ciclo completo iniciar → encuentro → finalizar, con entrega del bono;
  - compra en la cafetería usando ese bono.
- **Reservas:** el modelo rechaza una reserva en una fecha que ya pasó y acepta una en una fecha futura.
- **Persistencia:**
  - lo guardado se recupera en la siguiente ejecución;
  - un fallo al guardar deja los archivos sin cambios;
  - un archivo dañado se reporta con su nombre y no se sobrescribe;
  - si la carpeta no existe, se crean los datos de prueba;
  - si se acaba la entrada a mitad de un menú, se guarda antes de cerrar.

**12. Decisiones y supuestos del grupo**

| # | Decisión | Motivo |
|---|---|---|
| 1 | El torneo se crea con inscripciones abiertas. El administrador lo **inicia** (con al menos 2 inscritos), registra los encuentros y lo **finaliza** con las posiciones a mano. | El enunciado no dice cómo se juega el torneo. Así el historial tiene encuentros y posiciones. |
| 2 | El premio en efectivo es un % configurable del total recaudado (70 % por defecto) y es solo para el ganador. | "Un premio en efectivo acorde con el total recaudado". |
| 3 | Si el ganador del competitivo es empleado, el premio pasa al siguiente puesto que no sea de empleado. | Los empleados no reciben premio en efectivo. |
| 4 | El administrador define el % del bono al crear un torneo recreativo. Lo recibe cada integrante del primer puesto, empleados incluidos. | El enunciado solo excluye a los empleados del premio en efectivo. |
| 5 | El bono es de un solo uso y no se acumula con el código de empleado, con los puntos ni con otro bono. La mensualidad no lo admite. | "No acumulable con otros descuentos". |
| 6 | Ranking individual: el ELO de la academia. Ranking de un equipo: los puntos en partidos internos. "Los mejores" son los N primeros, con N = cupos preferenciales. | Usa información que el sistema ya tenía en el Proyecto 1. |
| 7 | Los externos y los estudiantes crean su propia cuenta como ParticipanteExterno. No reservan, no entran a equipos y no acumulan puntos. Declaran su nivel por disciplina. | Resuelve quién los registra y cómo se valida su nivel. |
| 8 | Un equipo lo inscribe un socio activo del equipo o su entrenador. | HU-TOR-SOC-01 CA-2. |
| 9 | El torneo se juega en un solo día. Un empleado está "de turno" si tiene un turno ese día de la semana. Una inscripción es "de empleado" (tarifa 0) si **todos** sus integrantes son empleados. | Simplifica HU-TOR-EMP-01 sin perder la regla. |
| 10 | El pádel siempre es en dobles. El nombre del torneo es único. La fecha límite de los preferenciales es anterior al inicio. | Reglas del Proyecto 1 y de las historias. |
| 11 | Los datos se guardan al terminar el programa, en dos pasos. | El enunciado no exige concurrencia. Guardar en dos pasos evita archivos dañados. |